SISTEMAS OPERATIVOS MONOPUESTO (SMR 1.º)

Procesos, hilos y planificación de la CPU

Tipo Guía | **Nivel** 1 · Básico | **Tema** 03 | **Duración** 5 h | **Herramientas** H2 (CMD/PowerShell), H4 (terminal Linux)

> 🧭 **Antes de empezar.** No hace falta máquina virtual para la teoría: trabajarás en el equipo del aula con Windows. Para los apartados 7 y 8 y los ejercicios de terminal usa tu MV de Linux Mint (`Mint-SMR`).

Ahora mismo, en tu ordenador, hay **cientos de programas funcionando a la vez**: el navegador, la música, el antivirus, el reloj de la barra de tareas… Pero la CPU sólo puede hacer **una cosa en cada instante** por núcleo. ¿Cómo lo consigue? Repartiendo el tiempo en turnos tan rápidos que parece que todo va a la vez. En esta guía vas a descubrir **quién reparte esos turnos y cómo**.

> 🧩 **Analogía.** Un **programa** es una receta escrita en un libro. Un **proceso** es esa receta cocinándose ahora mismo en una hornilla, con sus ingredientes y su punto de cocción. La misma receta puede estar cocinándose en tres hornillas a la vez: tres procesos del mismo programa. El **sistema operativo es el jefe de cocina**: decide qué hornilla usa el fuego y durante cuánto tiempo, porque el fuego (la CPU) es uno solo.

🎯 **Qué vas a aprender**	Qué es un proceso, cómo se reparte la CPU y cómo ver y controlar procesos en Linux y en Windows
🧰 **Qué necesitas**	El equipo del aula (Windows) y tu MV `Mint-SMR`
⏱️ **Tiempo**	5 horas
📦 **Qué entregas**	La tarea «Diagramas de planificación resueltos» (Parte 4)

Tu misión, paso a paso:

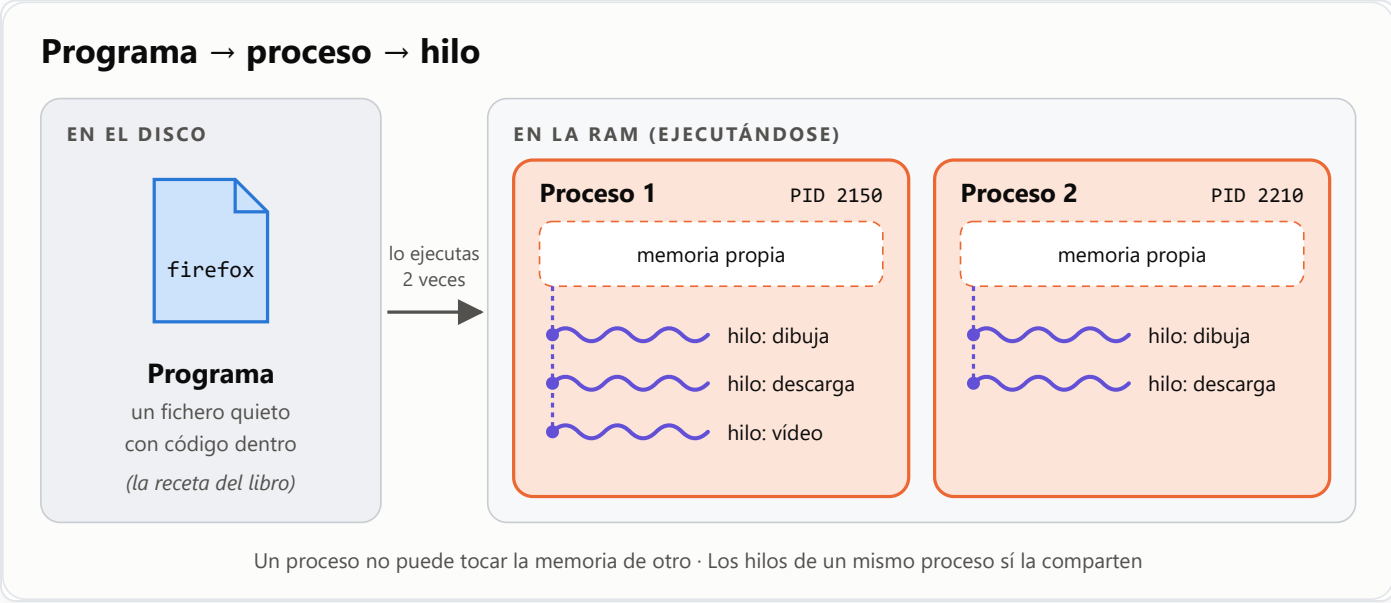
- ☐ Distinguir programa, proceso e hilo.
- ☐ Explicar los estados de un proceso con el diagrama.
- ☐ Dibujar diagramas de Gantt de FCFS, SJF y Round Robin y calcular tiempos de espera.
- ☐ Ver, buscar, pausar y terminar procesos en Linux y en Windows.
- ☐ Entregar los ejercicios resueltos.

Índice

- Procesos, hilos y planificación de la CPU
  - Índice
  - Parte 1 · Aprende
    - 1. Programa, proceso e hilo
    - 2. Estados de un proceso
    - 3. Cambio de contexto
    - 4. Algoritmos de planificación
      - Ejemplo resuelto
    - 5. Prioridades
    - 6. Concurrencia e interbloqueo
    - 7. Ver procesos en Linux
      - `ps`: la foto fija
      - `top`: el vídeo en directo
      - `htop` y `btop`: la versión con colores
    - 8. Gestionar procesos en Linux
      - Terminar un proceso: las señales
      - Primer y segundo plano
    - 9. Procesos en Windows
      - Administrador de tareas
      - Por línea de comandos
      - Demostración en clase
  - Parte 2 · Chuleta
  - Parte 3 · Practica
  - Parte 4 · Entrega

Parte 1 · Aprende

1. Programa, proceso e hilo



Concepto	Qué es	Dónde vive	En la cocina
Programa	Un fichero ejecutable: código guardado, quieto	En el disco	La receta del libro
Proceso	Una ejecución de ese programa, con su memoria propia	En la RAM	La receta cocinándose en una hornilla
Hilo	Una línea de trabajo dentro de un proceso	Comparte la memoria de su proceso	Dos cocineros en la misma hornilla: uno remueve y otro corta

Cada proceso lleva encima una especie de **DNI**:

- **PID** (*Process ID*): su número, único en todo el sistema.
- **PPID** (*Parent PID*): el PID del proceso que lo creó, su «padre».
- **Memoria propia**, aislada de la de los demás procesos.
- **Usuario propietario**, que decide qué puede hacer y qué no.
- **Estado**: si está trabajando, esperando… (apartado 2).

> ℹ️ **Nota**
>
> **¿Por qué existen los hilos?** Crear un proceso es caro: hay que reservarle memoria y prepararlo todo. Un hilo es mucho más barato porque **comparte** la memoria de su proceso. Por eso un navegador usa muchos hilos: uno dibuja la página, otro descarga, otro reproduce el vídeo. El precio de compartir es que, si dos hilos escriben lo mismo a la vez, el resultado es impredecible.

> 💡 **Consejo**
>
> **Pruébalo en Linux.** El terminal también es un proceso. Mira su PID y quién es su padre:
>
> ```
> echo $$                  # el PID de tu terminal (bash)
> ps -o pid,ppid,cmd       # tus procesos con su PID y el de su padre
> ```
>
> Fíjate en que el PPID de `ps` es **el PID de tu bash**: el terminal es el padre de todo lo que lanzas desde él.

🧠 ¿Lo has pillado?

1. Abres tres ventanas del mismo navegador. ¿Cuántos programas hay? ¿Y procesos?
2. ¿Por qué dos hilos pueden estropear un dato y dos procesos no?

▼ Ver respuestas

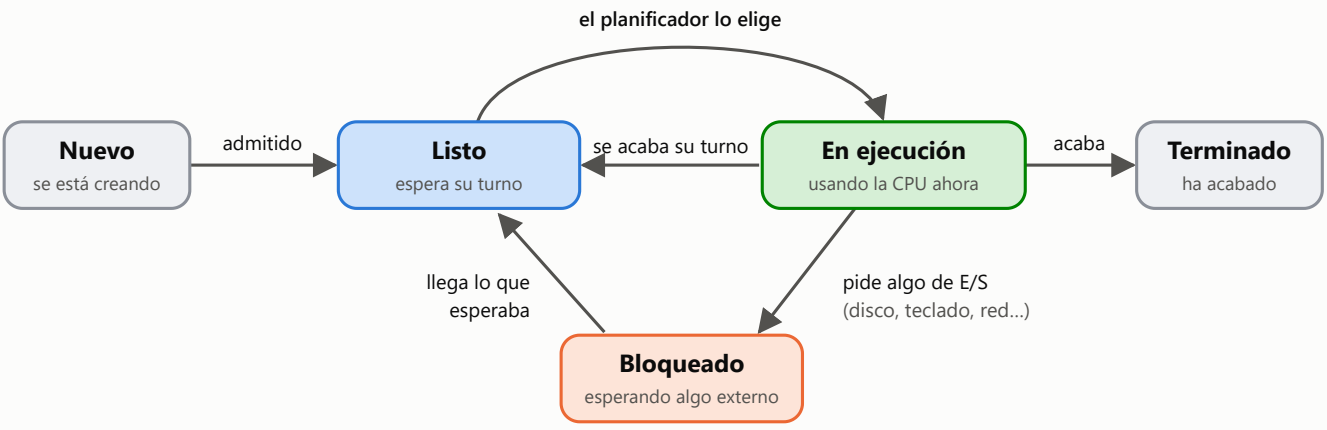
1. **Un programa** (el fichero del navegador) y **varios procesos**. Los navegadores modernos, además, crean un proceso por pestaña, así que verás más de tres.
2. Porque los hilos **comparten** la memoria y pueden escribir en el mismo sitio a la vez. Los procesos tienen cada uno **su memoria aislada**.

2. Estados de un proceso

Un proceso no está siempre trabajando. La mayor parte del tiempo **espera**: su turno, una tecla, un dato del disco…

Los estados de un proceso

En cada núcleo de la CPU sólo UN proceso está «en ejecución». Todos los demás esperan.



Fíjate: desde «Bloqueado» no se vuelve directamente a la CPU. Primero pasa a «Listo» y espera su turno.

Estado	Qué significa	Letra en Linux (`ps`, `top`)
Nuevo	Se está creando	—
Listo	Podría trabajar ya, pero la CPU está ocupada con otro	`R` (sí, la misma que en ejecución)
En ejecución	Está usando la CPU **ahora mismo**	`R`
Bloqueado	Espera algo externo: un dato del disco, una tecla, la red	`S` (durmiendo) o `D` (esperando al disco)
Terminado	Ha acabado; sólo queda recoger su código de salida	`Z` si nadie lo recoge (zombi)

> ❗ **Importante**
>
> **En un núcleo, sólo un proceso está en ejecución en cada instante.** Los cientos de procesos que ves en el Administrador de tareas están casi todos «listos» o «bloqueados». La sensación de que todo va a la vez es un efecto óptico: el sistema cambia de proceso **miles de veces por segundo**.

Dos casos raros que conviene conocer:

- 🧟 **Proceso zombi.** Ya ha terminado, pero su padre todavía no ha recogido su código de salida, así que sigue ocupando una línea en la tabla de procesos. No gasta CPU ni memoria: sólo un número.
- 👶 **Proceso huérfano.** Su padre ha muerto antes que él. El sistema lo adopta: en Linux pasa a colgar de `systemd`, el proceso con **PID 1**.

🧠 ¿Lo has pillado?

1. Un proceso pide leer un fichero del disco. ¿A qué estado pasa? Cuando el dato llega, ¿vuelve directamente a la CPU?
2. ¿Por qué no tiene sentido intentar «matar» un zombi?

▼ Ver respuestas

1. Pasa a **Bloqueado**. Cuando llega el dato pasa a **Listo**, no a ejecución: tiene que esperar a que el planificador le dé turno.
2. Porque **ya está muerto**: ha terminado. Sólo queda su «ficha» esperando a que el padre la recoja. Es la pregunta del ejercicio 7.

3. Cambio de contexto

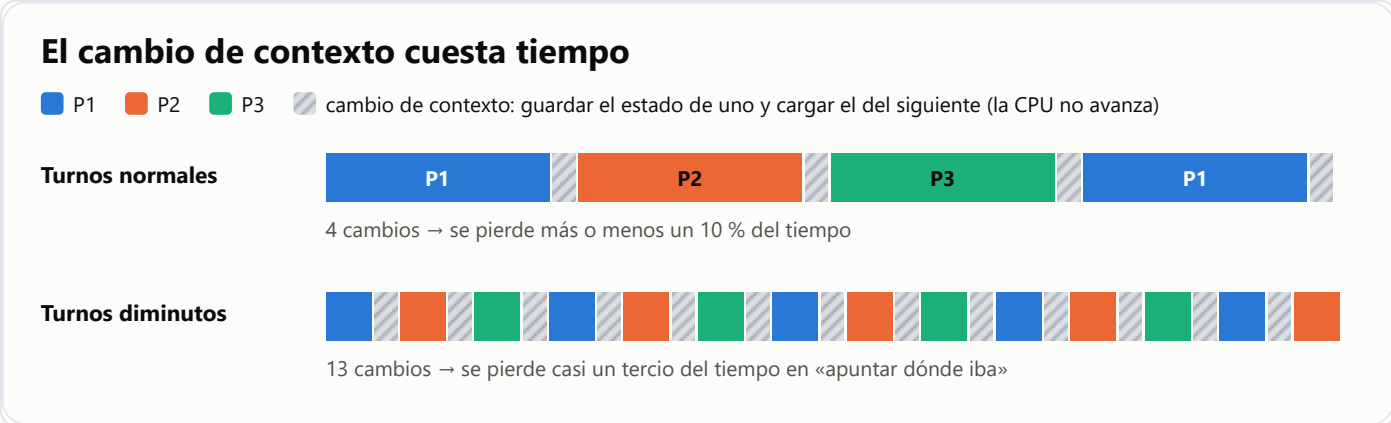
Cuando el sistema decide cambiar de proceso tiene que:

1. **Guardar** el estado del proceso actual: los registros de la CPU, por qué instrucción iba, su pila.

2. **Cargar** el estado del siguiente proceso.
3. Darle el control.

Ese estado guardado se llama **contexto**, y cambiarlo es **tiempo perdido**: mientras tanto la CPU no avanza en el trabajo de nadie.

> 🧩 **Analogía.** Estás resolviendo un problema de matemáticas y suena el móvil. Antes de cogerlo apuntas por dónde ibas; al colgar, releer tus notas y retomar el hilo te cuesta un rato. Ese rato es el cambio de contexto. **Cuantas más llamadas, menos matemáticas.**



Por eso los turnos (el **cuanto** del apartado 4) no pueden ser diminutos: si el turno dura casi lo mismo que el cambio de contexto, el ordenador se pasa el día apuntando dónde iba.

> ℹ️ **Actividad de clase: «somos la CPU».** Una persona hace de CPU; el resto sois procesos con una tarea escrita (sumar una columna de números). La CPU sólo puede atender a uno y cada cierto tiempo tiene que cambiar, apuntando por dónde iba cada uno. Se cronometra cuánto tarda el grupo con turnos de **20 s**, de **5 s** y de **60 s**. ¿Cuál gana? ¿Por qué?

4. Algoritmos de planificación

El **planificador** es el árbitro del sistema operativo: decide **qué proceso listo pasa a la CPU**. Hay muchas formas de decidirlo; estas son las tres clásicas.

Algoritmo	Cómo decide	👍 Ventaja	👎 Problema	En la vida real
FCFS (*First Come, First Served*)	Por orden de llegada, sin interrumpir	Muy simple y justo	Si el primero es larguísimo, todos esperan	La cola del súper

Algoritmo	Cómo decide	👍 Ventaja	👎 Problema	En la vida real
SJF (*Shortest Job First*)	Primero el que menos tiempo necesita	El mejor tiempo medio de espera	Hay que **adivinar** cuánto durará cada uno, y los largos pueden no pasar nunca (**inanición**)	«Pasa tú, que sólo llevas una barra de pan»
Round Robin	Turnos fijos (**cuanto**); si no acabas, vuelves al final de la cola	Nadie espera eternamente	Si el cuanto es pequeño, mucho cambio de contexto	Turnos de palabra de un minuto en una asamblea

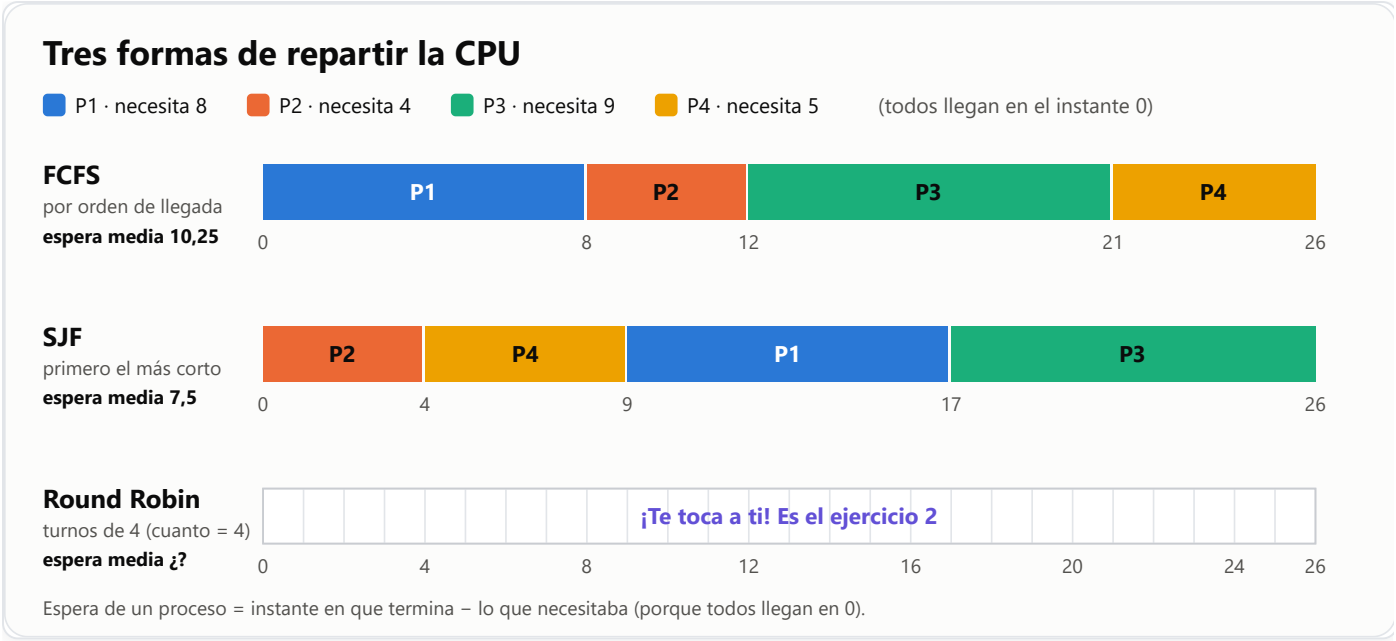
> ℹ️ **Nota**
>
> Los sistemas que usas (Windows, Linux, macOS) son **interactivos** y usan variantes de Round Robin con prioridades: lo importante es que el ratón y el teclado respondan rápido, aunque las tareas largas tarden un poco más.

Ejemplo resuelto

Cuatro procesos que **llegan todos en el instante 0**:

Proceso	Ráfaga de CPU (lo que necesita)
P1	8
P2	4
P3	9
P4	5

Así quedan los **diagramas de Gantt** (una barra de tiempo con quién usa la CPU en cada momento):



Dos medidas para comparar algoritmos:

- **Tiempo de retorno**: el instante en que el proceso **termina** (desde que llegó).
- **Tiempo de espera**: cuánto ha estado esperando sin usar la CPU = retorno − ráfaga.

FCFS (orden P1, P2, P3, P4):

Proceso	Empieza	Termina = retorno	Espera
P1	0	8	8 − 8 = **0**
P2	8	12	12 − 4 = **8**
P3	12	21	21 − 9 = **12**
P4	21	26	26 − 5 = **21**

Tiempo medio de espera = (0 + 8 + 12 + 21) / 4 = **10,25**

SJF (del más corto al más largo: P2, P4, P1, P3):

Proceso	Empieza	Termina = retorno	Espera
P2	0	4	4 − 4 = **0**
P4	4	9	9 − 5 = **4**
P1	9	17	17 − 8 = **9**
P3	17	26	26 − 9 = **17**

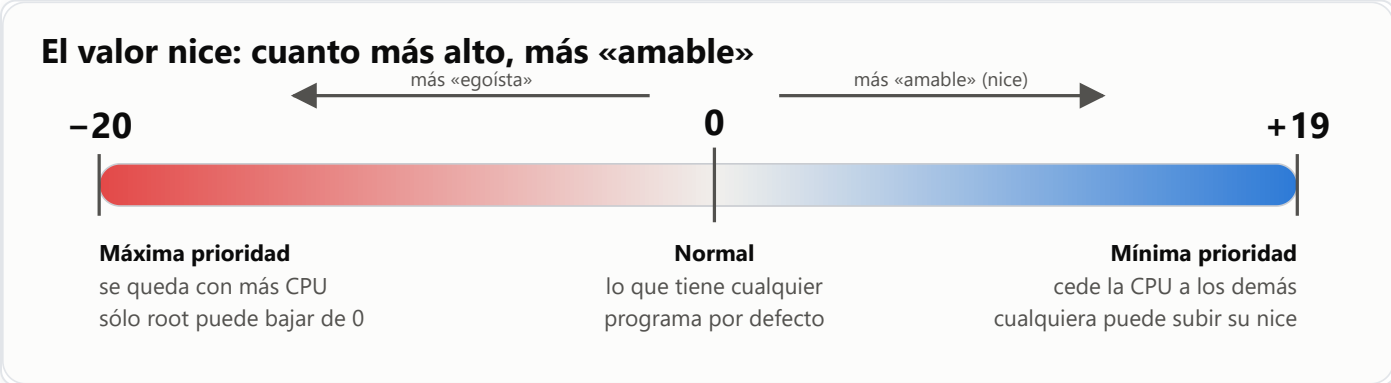
Tiempo medio de espera = (0 + 4 + 9 + 17) / 4 = **7,5**

> 💡 **Consejo**
>
> **Método para cualquier Gantt.** 1) Decide el orden según el algoritmo. 2) Dibuja la barra sumando ráfagas y apunta debajo cada instante de cambio. 3) Saca la tabla: retorno = cuándo termina; espera = retorno − ráfaga (si todos llegan en 0). 4) Haz la media.

Round Robin con cuanto = 4: dibújalo tú. Es el ejercicio 2.

5. Prioridades

No todos los procesos valen lo mismo: que suene la música sin cortes importa más que terminar una copia de seguridad un minuto antes. En Linux la prioridad se expresa con el valor `nice` («amable» en inglés):



Valor `nice`	Significado
−20	Máxima prioridad: el proceso más «egoísta»
0	Normal (el valor por defecto)
+19	Mínima prioridad: el más «amable», cede la CPU a los demás

```
nice -n 10 comando          # lanzar un comando con prioridad baja
renice -n 15 -p 1234        # bajar la prioridad de un proceso tuyo ya lanzado
sudo renice -n -5 -p 1234   # subirla (valor negativo): sólo root
```

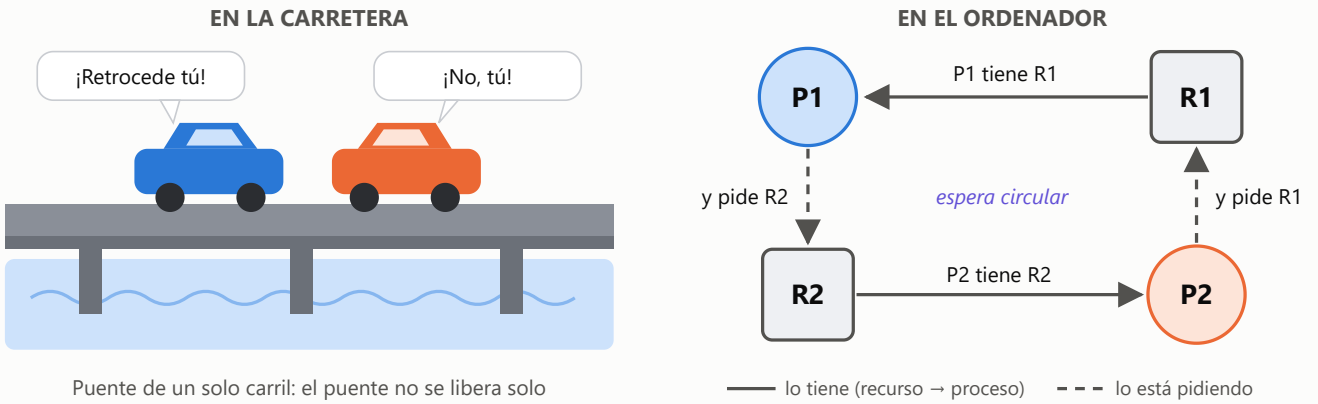
> ❗ **Importante**
>
> **Sólo root puede subir la prioridad** (valores negativos). Bajarla, cualquiera. Si no fuera así, todo el mundo se pondría a −20 y volveríamos al punto de partida.

En `top` y `htop` lo ves en la columna `NI`. En `htop` puedes cambiarlo con `F7` y `F8`.

6. Concurrencia e interbloqueo

Cuando varios procesos compiten por los mismos recursos (un fichero, la impresora, una zona de memoria) pueden quedarse **bloqueados para siempre esperándose unos a otros**. Eso es un **interbloqueo** (*deadlock*).

Interbloqueo: todos esperan, nadie avanza



Para que ocurra hacen falta **cuatro condiciones a la vez** (las **condiciones de Coffman**):

Condición	Qué significa	En el puente
1. **Exclusión mutua**	El recurso no se puede compartir	En el carril sólo cabe un coche
2. **Retención y espera**	Quien tiene un recurso pide otro sin soltar el que tiene	Cada coche ocupa su mitad y quiere la otra
3. **Sin expropiación**	No se le puede quitar el recurso por la fuerza	No hay grúa que aparte un coche
4. **Espera circular**	A espera a B, B espera a A (o a C, y C a A…)	Cada uno espera a que el otro retroceda

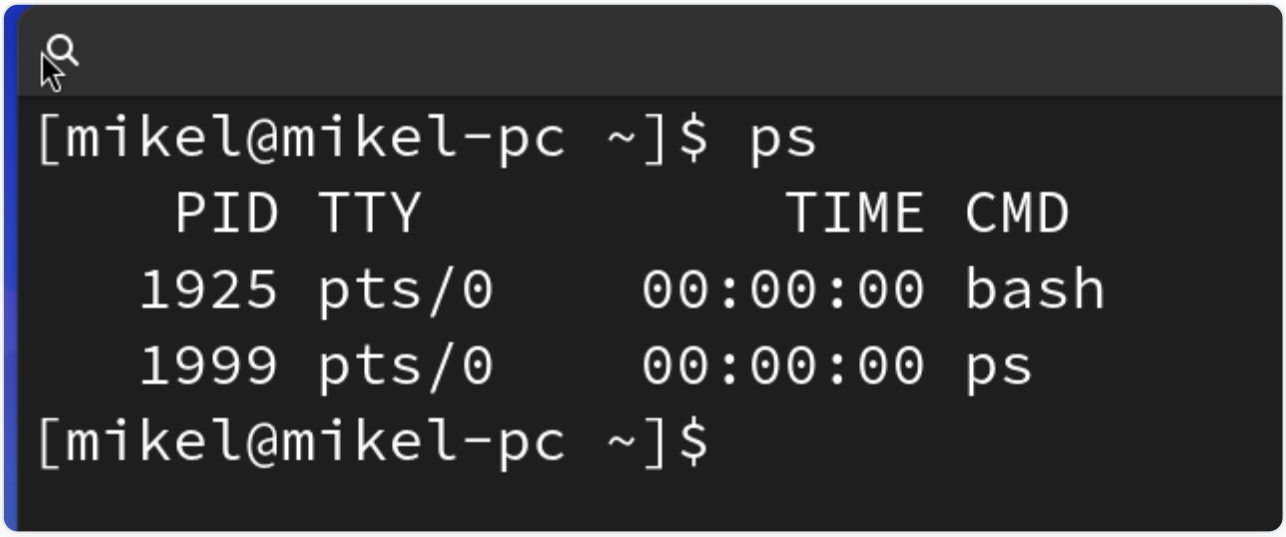
> 💡 **Consejo**
>
> **Rompiendo cualquiera de las cuatro, el interbloqueo es imposible.** Por ejemplo: un semáforo en el puente rompe la exclusión mutua «a lo bruto» (sólo entra un sentido cada vez). A este nivel basta con saber identificar las cuatro condiciones.

7. Ver procesos en Linux

`ps`: la foto fija

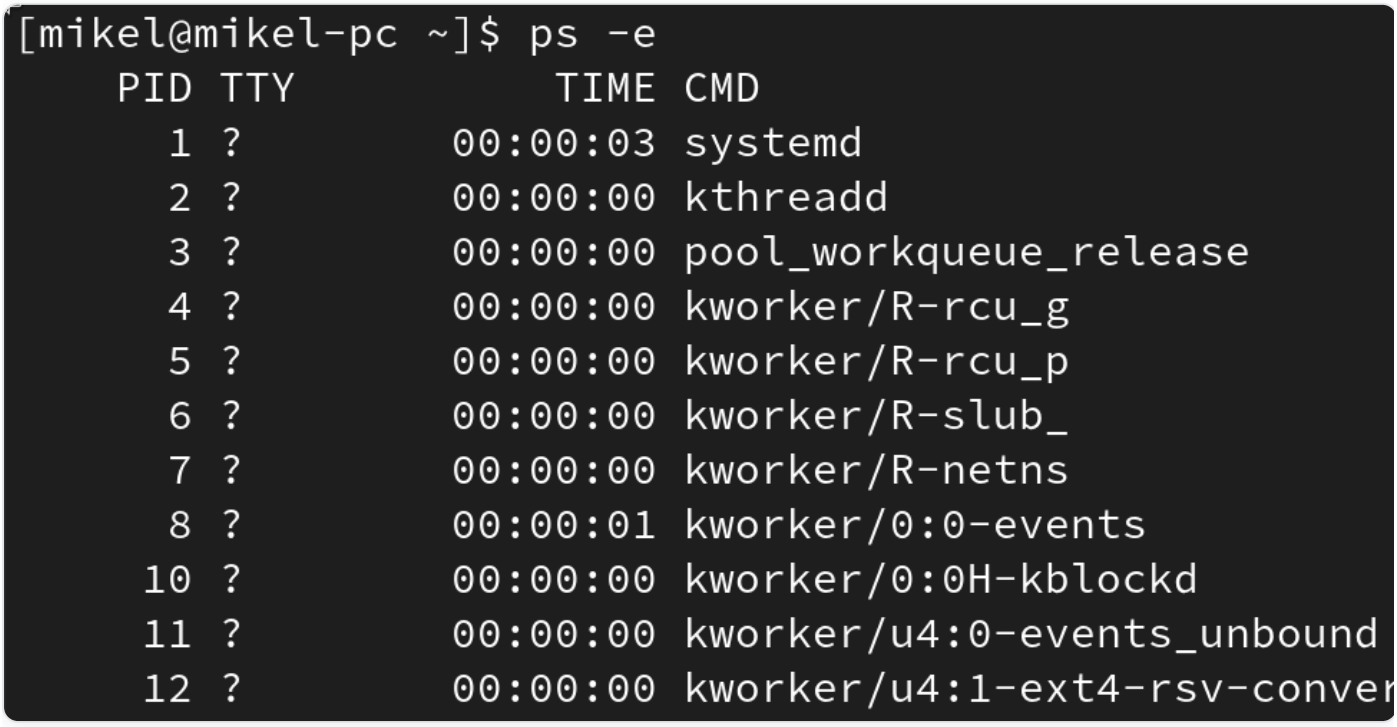
`ps` (*process status*) hace una **foto** de los procesos en ese instante.

```
ps
```



Sin opciones sólo enseña **los procesos de tu terminal**. Casi nunca es lo que quieres.

```
ps -e
```



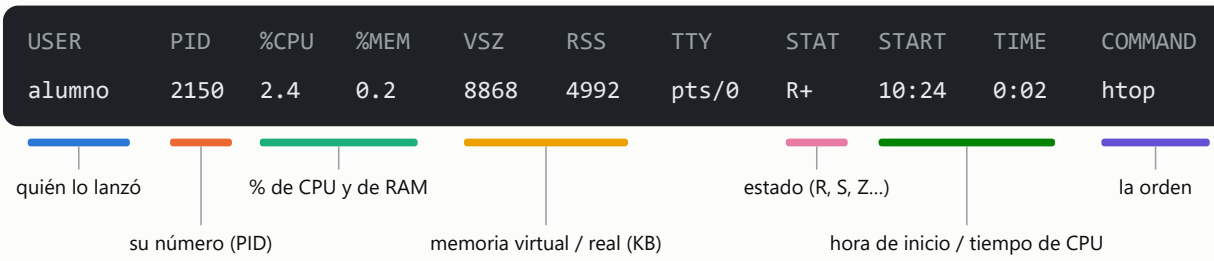
Todos los procesos del sistema, en formato corto. Fíjate en el primero: `systemd`**, PID 1**, el padre de todos.

```
ps aux
```

```
[mikel@mikel-pc ~]$ ps -aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.1  0.6  22712 13728 ?        Ss   07:11   0:03 /sbin/init
root           2  0.0  0.0      0     0 ?        S    07:11   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    07:11   0:00 [pool_workqueue_relea
root           4  0.0  0.0      0     0 ?        I<   07:11   0:00 [kworker/R-rcu_g]
root           5  0.0  0.0      0     0 ?        I<   07:11   0:00 [kworker/R-rcu_p]
```

El más usado. Así se lee cada línea:

Cómo leer una línea de «ps aux»



Letra en `STAT`	Estado
`R`	*Running*: ejecutándose o listo para hacerlo
`S`	*Sleeping*: durmiendo, esperando algo (lo más habitual)
`D`	Esperando al disco; no se puede interrumpir
`T`	Detenido (por ejemplo, con `Ctrl` + `Z`)
`Z`	Zombi
`I`	Hilo del núcleo sin trabajo (*idle*)

Los símbolos que acompañan a la letra: `<` prioridad alta, `N` prioridad baja, `s` jefe de sesión, `+` en primer plano del terminal.

Combinaciones útiles:

```
ps aux | grep firefox                  # buscar un proceso por nombre
ps -ef --forest                        # árbol de procesos padre → hijo
ps aux --sort=-%mem | head -n 11       # los 10 que más memoria consumen
ps -u alumno                           # procesos de un usuario
pgrep -l firefox                       # sólo el PID y el nombre
```

> 💡 **Consejo**
>
> En `ps aux | grep firefox` siempre aparece una línea de más: ¡el propio `grep firefox`! `pgrep` no tiene ese problema.

`top`: el vídeo en directo

`top` (*table of processes*) se **actualiza solo** cada pocos segundos.

```
top
```

```
top - 07:38:27 up 26 min,  1 user,  load average: 0,48, 0,35, 0,26
Tareas: 186 total,   1 ejecutar,  185 hibernar,    0 detener,    0 zombie
%Cpu(s): 12,6 us, 14,8 sy,  0,0 ni, 59,9 id,  0,0 wa,  0,0 hi, 12,6 si,  0,0 st
MiB Mem :   1955,5 total,    190,0 libre,   1265,8 usado,    667,6 búf/caché
MiB Intercambio:   7182,0 total,   6778,1 libre,    403,9 usado.    689,6 dispon Mem

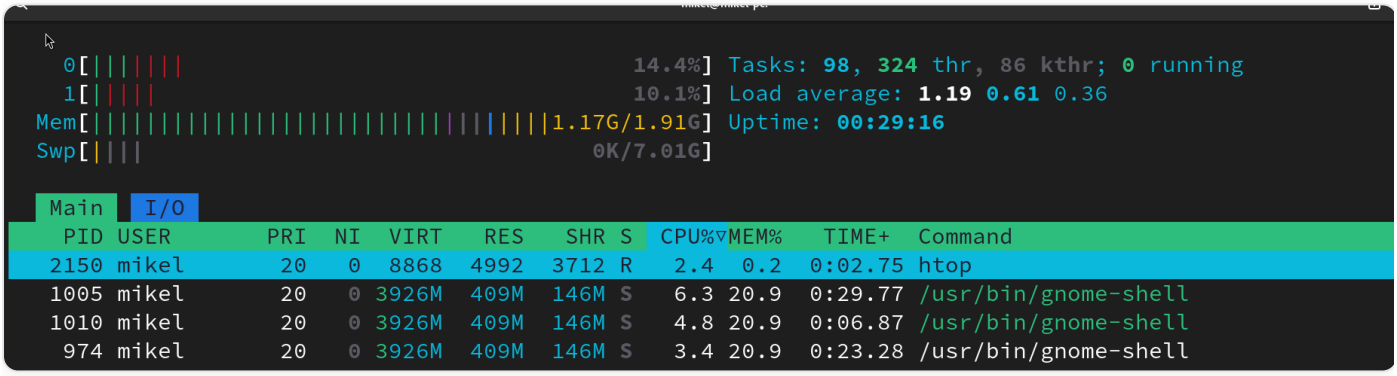
    PID USUARIO    PR  NI    VIRT    RES    SHR S  %CPU  %MEM     HORA+ ORDEN
    974 mikel      20   0 3989108 393060 146524 S  47,4  19,6   1:00.97 gnome-shell
   1873 mikel      20   0 1632892 316320 139756 S  30,1  15,8   1:01.49 kgx
   2025 root       20   0       0      0      0 I   2,6   0,0   0:00.46 kworker/u5:1-event+
      8 root       20   0       0      0      0 I   1,0   0,0   0:03.61 kworker/0:0-md
     17 root       20   0       0      0      0 S   1,0   0,0   0:01.87 ksoftirqd/0
     18 root       -2   0       0      0      0 I   0,3   0,0   0:01.06 rcu_preempt
     20 root       rt   0       0      0      0 S   0,3   0,0   0:00.06 migration/0
     27 root       20   0       0      0      0 I   0,3   0,0   0:01.87 kworker/1:0-mm_per+
     29 root       20   0       0      0      0 I   0,3   0,0   0:01.13 kworker/u5:0-event+
```

Arriba tienes el resumen (cuántas tareas hay, cuánta CPU y memoria se usa); abajo, la lista. Teclas dentro de `top`:

Tecla	Qué hace
`M`	Ordenar por memoria
`P`	Ordenar por CPU
`k`	Matar un proceso (te pide el PID)
`u`	Filtrar por usuario
`1`	Ver cada núcleo por separado
`q`	Salir

`htop` y `btop`: la versión con colores

```
sudo apt install htop btop   # Linux Mint / Ubuntu
sudo pacman -S htop btop     # Arch
htop
```



Interactivos, con colores y con ratón. `btop` es la versión más moderna y añade gráficas de disco y red. Las teclas de `htop` aparecen abajo en la pantalla:

Tecla	Qué hace
`F3`	Buscar un proceso
`F5`	Ver el árbol padre → hijo
`F6`	Elegir columna para ordenar
`F7` / `F8`	Subir / bajar la prioridad (`nice`)
`F9`	Enviar una señal (matar)
`F10` o `q`	Salir

8. Gestionar procesos en Linux

Terminar un proceso: las señales

Para terminar un proceso no se le «mata» directamente: se le **envía una señal**, un aviso del sistema.

```
kill 1234                    # pedir que termine (SIGTERM, la educada)
kill -9 1234                 # forzar (SIGKILL, la brusca)
killall firefox              # por nombre exacto del programa
pkill -f "python mi_script.py"   # por coincidencia con la orden completa
```

Señal	Número	Qué hace	Analogía
`SIGTERM`	15	«Termina cuando puedas». El programa cierra sus ficheros y guarda	Llamar a la puerta
`SIGKILL`	9	El núcleo lo elimina en seco. **No se puede ignorar** y no da tiempo a guardar	Tirar la puerta abajo
`SIGSTOP`	19	Lo congela	Darle a pausa
`SIGCONT`	18	Lo reanuda	Darle a *play*

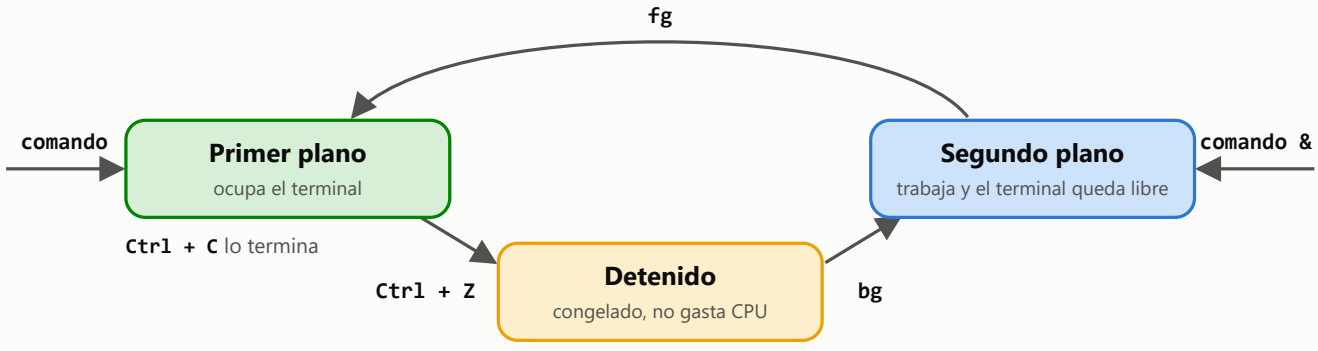
> ⚠️ **Aviso**
>
> **Empieza siempre por `kill` a secas.** `kill -9` es como desenchufar el ordenador: funciona siempre, pero puede dejar ficheros a medio escribir. Úsalo sólo si el proceso ignora la señal educada.

> ⚠️ **Aviso**
>
> **Error típico:** escribir `killid` o `kill id`. El comando es `kill` seguido del **PID** (un número): `kill 1234`.

Primer y segundo plano

Cuando lanzas un comando, el terminal se queda «ocupado» hasta que acaba: está en **primer plano**. Puedes mandarlo a trabajar **en segundo plano** y seguir usando el terminal.

Primer y segundo plano en el terminal



jobs lista los trabajos de este terminal · **fg** también despierta a uno detenido · **nohup comando &** sobrevive al cerrar el terminal

```
comando &          # lanzar directamente en segundo plano
Ctrl + Z           # detener (pausar) el proceso en primer plano
jobs               # listar los trabajos de este terminal
bg                 # reanudar el detenido en segundo plano
fg                 # traerlo a primer plano
nohup comando &    # que siga vivo aunque cierres el terminal
```

> 🖍️ **Ejemplo.** Pruébalo tal cual en tu MV:
>
> ```
> sleep 100          # el terminal se queda bloqueado…
> # pulsa Ctrl + Z   → [1]+  Detenido   sleep 100
> jobs               # [1]+  Detenido   sleep 100
> bg                 # [1]+ sleep 100 &   (sigue contando, y el terminal queda libre)
> fg                 # vuelve a primer plano
> # pulsa Ctrl + C   → lo termina
> ```

9. Procesos en Windows

Administrador de tareas

Se abre con `Ctrl` + `Shift` + `Esc`. Pestañas que más vas a usar:

Pestaña	Para qué
Procesos	Los procesos agrupados por aplicación, con CPU, memoria, disco y red
Rendimiento	Las gráficas globales de CPU, memoria, disco y red
Detalles	La vista técnica, **con el PID**. Es la que usaremos en clase
Servicios	Los procesos del sistema que funcionan sin ventana

Clic derecho sobre un proceso en **Detalles**:

- **Establecer prioridad** → el equivalente a `nice`.
- **Finalizar tarea** → el equivalente a `kill`.

Por línea de comandos

CMD:

```
tasklist                                  :: listar procesos
tasklist /FI "IMAGENAME eq notepad.exe"   :: filtrar por nombre
taskkill /PID 1234                        :: terminar por PID
taskkill /IM notepad.exe /F               :: terminar por nombre, forzando
```

PowerShell:

```
Get-Process                                    # listar
Get-Process | Sort-Object CPU -Descending | Select-Object -First 10
Get-Process notepad                            # buscar por nombre
Stop-Process -Id 1234                          # terminar
Stop-Process -Name notepad -Force              # forzar
Get-Process | Where-Object {$_.WS -gt 200MB}   # los que usan más de 200 MB
(Get-Process notepad).PriorityClass = 'BelowNormal' # bajar la prioridad
```

Demostración en clase

1. Abre el Administrador de tareas en la pestaña **Detalles** y ordena por memoria.
2. Abre un editor de imágenes y carga una imagen grande (varios megapíxeles).
3. Observa cómo sube la memoria del proceso.
4. Aplica un filtro pesado (desenfoque, escalado) y observa el pico de CPU.
5. Apunta los valores en esta tabla. **Es parte del entregable.**

Momento	Memoria del proceso	% de CPU
Editor abierto, sin imagen		
Con la imagen cargada		
Aplicando el filtro		

Parte 2 · Chuleta

Tenla a mano mientras haces los ejercicios.

Linux ↔ Windows

Acción	Linux	Windows (CMD)	Windows (PowerShell)
Listar procesos	`ps aux`	`tasklist`	`Get-Process`
Ver en tiempo real	`top` / `htop`	Administrador de tareas	Administrador de tareas
Buscar por nombre	`pgrep -l nombre`	`tasklist /FI "IMAGENAME eq nombre.exe"`	`Get-Process nombre`
Terminar (educado)	`kill PID`	`taskkill /PID n`	`Stop-Process -Id n`

Acción	Linux	Windows (CMD)	Windows (PowerShell)
Terminar (forzado)	`kill -9 PID`	`taskkill /PID n /F`	`Stop-Process -Id n -Force`
Cambiar prioridad	`renice -n 10 -p PID`	`start /low programa` (al lanzarlo)	`(Get-Process nombre).PriorityClass = 'BelowNormal'`
Ver el árbol	`ps -ef --forest`	—	—
Identificador	PID	PID	Id

Estados (columna `STAT`)

Letra	Estado
`R`	Ejecutando o listo
`S`	Durmiendo (esperando algo)
`D`	Esperando al disco
`T`	Detenido
`Z`	Zombi

Señales

Señal	Nº	Efecto	Cómo enviarla
`SIGTERM`	15	Termina educadamente	`kill PID`
`SIGKILL`	9	Termina a la fuerza	`kill -9 PID`
`SIGSTOP`	19	Pausa	`kill -STOP PID` o `Ctrl` + `Z`*
`SIGCONT`	18	Reanuda	`kill -CONT PID`, `fg` o `bg`

* `Ctrl` + `Z` envía en realidad `SIGTSTP`, la versión «educada» de `SIGSTOP`.

Planificación

Algoritmo	Regla de oro
FCFS	El que llega primero, primero. Sin interrupciones
SJF	El más corto primero. Mejor espera media, pero hay que adivinar la duración

Algoritmo	Regla de oro
Round Robin	Turnos de un cuanto; si no acabas, al final de la cola

Espera = retorno − ráfaga (si todos llegan en 0) · Espera media = suma de esperas / nº de procesos

Glosario exprés

Palabra	Significado
PID / PPID	Número del proceso / número de su padre
Hilo	Línea de trabajo dentro de un proceso; comparte su memoria
Planificador	La parte del SO que decide quién usa la CPU
Cuanto	El tiempo máximo de cada turno en Round Robin
Contexto	El estado de un proceso que hay que guardar y cargar al cambiar de turno
Ráfaga	El tiempo de CPU que necesita un proceso
Inanición	Un proceso que nunca consigue turno
Interbloqueo	Varios procesos esperándose entre sí para siempre
`nice`	Valor de prioridad en Linux: de −20 (máxima) a +19 (mínima)

Parte 3 · Practica

> 💡 **Consejo**
>
> Los ejercicios 2, 3 y 7 forman parte del entregable. El resto, hazlos igualmente: son los que te preparan para la prueba práctica.

Ejercicio 1 · Cuenta procesos. Abre `htop` en tu MV y responde: ¿cuántos procesos hay en total (*Tasks*)? ¿Cuántos están en ejecución (*running*, estado `R`)? ¿Por qué la diferencia es tan grande? *(Pista: vuelve a la caja «Importante» del apartado 2.)*

Ejercicio 2 · Round Robin. Dibuja el diagrama de Gantt de **Round Robin con cuanto = 4** para los procesos del ejemplo resuelto y calcula el tiempo medio de espera. Compáralo con FCFS (10,25) y con SJF (7,5). ¿Cuál es mejor? ¿Y si los procesos fueran interactivos, como un juego o un editor de texto?

> 💡 **Consejo**
>
> **Cómo empezar.** El orden de la cola es P1, P2, P3, P4. A P1 le tocan 4 unidades (de 0 a 4) y, como necesitaba 8, le quedan 4: vuelve **al final** de la cola. Luego P2 (de 4 a 8), que necesitaba justo 4 y **termina**… Sigue tú. Ve apuntando lo que le queda a cada uno:

Turno	Proceso	De… a…	Le queda
1	P1	0 → 4	4
2	P2	4 → 8	0 ✓
3			

Ejercicio 3 · Con tiempos de llegada. Ahora los procesos **no** llegan todos a la vez. Resuelve FCFS y SJF **no expropiativo** (el que está en la CPU no se interrumpe aunque llegue uno más corto):

Proceso	Llegada	Ráfaga
A	0	6
B	2	3
C	4	8
D	5	2

> ❗ **Importante**
>
> **Con llegadas, la espera cambia:** espera = instante en que termina − llegada − ráfaga. Y en SJF sólo puedes elegir entre los procesos **que ya han llegado** cuando la CPU queda libre. En el instante 0 sólo está A, así que empieza A aunque no sea el más corto.

Ejercicio 4 · Tres formas de matar. Lanza `sleep 300 &` tres veces. Localiza los tres PID con `ps` y termínalos de tres formas distintas: por PID (`kill`), por nombre (`killall`) y con `pkill` . Copia los comandos y lo que responde el terminal.

Ejercicio 5 · El proceso glotón. Lanza `yes > /dev/null &` (un programa que escribe «y» sin parar a la papelera). Mira su consumo de CPU en `htop` . Bájale la prioridad con `renice -n 19 -p PID` y observa si cambia algo. Termínalo.

> ℹ️ **Nota**
>
> Si tu MV tiene varios núcleos, puede que **no veas diferencia**: `yes` tiene un núcleo para él solo y no compite con nadie. Mira cuántos núcleos tienes con `nproc` y lanza **un `yes` más que núcleos**. Ahora sí compiten: ¿cuál recibe menos CPU?

Ejercicio 6 · El que más memoria gasta. Encuentra el proceso que más memoria consume en tu equipo con **un solo comando** en Linux y con **un solo comando** en PowerShell. *(Pista: en la chuleta y en el apartado 7 tienes cómo ordenar.)*

Ejercicio 7 · Zombis. Explica con tus palabras qué es un proceso zombi y por qué no se puede «matar» con `kill -9`. ¿Qué habría que hacer para que desapareciera?

Parte 4 · Entrega

Tarea Moodle: «Diagramas de planificación resueltos» Calificación de 0 a 10 · Se aprueba con 5.

Entrega un documento con:

1. **Ejercicios 2, 3 y 7 resueltos**, con los diagramas de Gantt dibujados (a mano y fotografiados, o con cualquier programa) y los tiempos calculados en tabla.
2. **Informe de proceso pesado**: captura del Administrador de tareas o de `htop` con el proceso que más consume en tu equipo, indicando **PID, usuario, CPU y memoria**, y explicando qué es ese proceso y si es normal que consuma eso.
3. La **tabla de medidas** de la demostración del apartado 9.
4. La **tabla de equivalencias** de la chuleta rellenada **de memoria** en una hoja y luego corregida (entrega las dos versiones).

Antes de entregar, comprueba:

- ☐ Cada Gantt tiene los instantes de cambio escritos debajo.
- ☐ Cada tabla de tiempos tiene la espera de cada proceso **y** la media.
- ☐ En el ejercicio 2 dices cuál es el mejor algoritmo **y por qué**.
- ☐ El informe del proceso pesado tiene PID, usuario, CPU y memoria.
- ☐ Las respuestas están escritas con tus palabras.

Instrumento de evaluación: corrección de ejercicios y observación de la práctica.
